ЛР1. Теория: BI, хранилища данных, OLAP, модель «звезда»

Материал лекций «Оперативный анализ данных», «Архитектура OLAP», «ETL» и вопросы РК1 № 16–30. В конце каждого раздела — что из этого видно в вашей ЛР1.

1. Зачем нужны BI и хранилище данных

OLTP-системы (учётные, транзакционные: касса, CRM, сайт) оптимизированы под короткие операции записи одной строки. **Аналитика** (OLAP) читает миллионы строк и агрегирует их. Если делать отчёты прямо в OLTP-базе, тяжёлые запросы мешают работе системы, а данные из разных источников не согласованы. Поэтому данные копируют в отдельное **хранилище данных** (Data Warehouse, DWH).

1.1. Четыре свойства хранилища (Инмон)

Свойство	Содержание
Предметная ориентированность	Данные организованы вокруг предметов анализа (продажи, клиенты, товары), а не вокруг приложений-источников.
Интегрированность	Данные из разных источников приведены к единым кодам, единицам, форматам и ключам («М/Ж» и «1/0» → одно представление).
Неизменчивость (non-volatile)	Данные загружаются и читаются; их не правят и не удаляют точечно. История не теряется.
Привязка ко времени (time-variant)	Каждая запись относится к моменту или периоду времени; хранилище хранит историю на годы.

1.2. Слои хранилища

Слой	Назначение
Staging (промежуточная зона)	«Сырые» копии данных источников, как есть. Временная зона для загрузки, проверок и сверок; аналитики её не читают.
ODS (оперативный склад)	Интегрированные, очищенные, но *текущие* данные с минимальной историей. Нужен для оперативной отчётности почти в реальном времени.
Core (ядро, DDS)	Интегрированная история: нормализованная модель (Инмон) или конформные измерения и факты (Кимбалл).
Data Marts (витрины)	Денормализованные наборы под конкретный бизнес-процесс или отдел; обычно схема «звезда». Отсюда читают BI-инструменты.

Отличие **staging от ODS**: staging не интегрирован, и его не читают пользователи; ODS интегрирован и доступен для оперативных запросов, но не хранит длинную историю.

1.3. Подходы Инмона и Кимбалла

- **Инмон** (top-down): сначала корпоративное хранилище в 3НФ — единый «источник правды», из него строятся витрины.
- **Кимбалл** (bottom-up): сразу строятся витрины-«звёзды» по бизнес-процессам. Целостность достигается **шиной конформных измерений** — общими справочниками (Время, Товар, Клиент),

которые одинаково используются всеми витринами.

Критерий	Инмон	Кимбалл
Направление	сверху вниз	снизу вверх
Модель ядра	нормализованная (3НФ)	многомерная (звёзды)
Срок до первого результата	долгий	короткий
Стоимость старта	высокая	низкая
Гибкость к новым вопросам	высокая	средняя (зависит от зерна)
Целостность данных	центральная модель	конформные измерения
Команда	большая, нужны архитекторы	меньше, ближе к бизнесу
Удобство для аналитиков	через витрины	сразу

1.4. Метаданные

Вид	Что описывает	Кто формирует
Технические	таблицы, типы, индексы, расписания загрузок	СУБД и ETL-инструменты автоматически
Бизнес-метаданные	смысл показателя, формула, владелец, допустимые значения	люди (аналитики, владельцы данных)
Операционные	время загрузки, число строк, ошибки, источник версии	система загрузки (логи ETL)

Чаще всего **отсутствуют бизнес-метаданные**: их нельзя сгенерировать автоматически. Последствие — разные отделы по-разному понимают «выручку» или «активного клиента», цифрам перестают доверять.

> **В ЛР1.** Ваш Power BI-файл — это маленькая витрина: CSV играет роль источника, Power Query — ETL, модель — «звезда».

2. Многомерная модель: куб, меры, измерения

2.1. Гиперкуб

Гиперкуб — логическая модель, в которой каждая **мера** (числовой показатель) рассматривается как функция от набора **измерений**:

$$\text{Продажи} = f(\text{Время}, \text{Товар}, \text{Регион}, \text{Клиент}).$$

Ячейка куба — значение меры при фиксированных значениях всех измерений. Куб называют **логической** моделью, потому что он описывает, *как пользователь думает о данных*, а не как они лежат на диске. Физически куб может храниться в реляционных таблицах (ROLAP), в многомерных массивах (MOLAP) или комбинированно (HOLAP).

2.2. Меры и измерения

- **Мера** — то, что агрегируют (считают, суммируют, усредняют): `Sales`, `Profit`, `Quantity`.
- **Измерение** — то, по чему группируют и фильтруют: дата, регион, категория.

Правило разграничения: поле — мера, если по нему осмысленно агрегировать; измерение — если по нему осмысленно группировать. **Тип данных роль не определяет:** почтовый индекс `Postal Code` числовой, но складывать индексы бессмысленно — это измерение. Год тоже число, но используется для группировки.

2.3. Гранулярность (зерно)

Зерно таблицы фактов — что означает одна её строка. В Superstore одна строка — это одна позиция заказа (товар в заказе).

Принцип невосстановимости детализации: из детальных данных всегда можно получить агрегат, а из агрегата детали — никогда. Поэтому зерно выбирают максимально детальным и объявляют **до** проектирования измерений и мер (правило Кимбалла). Смешение зёрен в одной таблице (позиции и итоги по заказу в одном столбце) даёт двойной счёт.

2.4. Аддитивность мер

Класс	Определение	Примеры
Аддитивные	суммируются по всем измерениям	выручка, количество, себестоимость
Полуаддитивные	суммируются по всем, кроме времени	остаток на складе, баланс счёта, число сотрудников на дату
Неаддитивные	не суммируются вообще	цена, рентабельность, средний чек, процент скидки

Математика ошибки «усреднения средних». Пусть в группах $k = 1, \dots, K$ прибыль P_k и продажи S_k. Верная рентабельность группы:

$$r = \frac{\sum_k P_k}{\sum_k S_k} = \sum_k \frac{S_k}{\sum_j S_j}\, r_k, \qquad r_k = \frac{P_k}{S_k},$$

то есть это **взвешенное** среднее рентабельностей с весами — долями продаж. Простое среднее $\frac{1}{K}\sum_k r_k$ совпадает с r только при равных S_k. Поэтому неаддитивные меры хранят в виде числителя и знаменателя, а делят в момент запроса — как ваша мера `DIVIDE(SUM(Profit), SUM(Sales))`.

2.5. Иерархии

Вид	Описание	Пример
Сбалансированная	все ветви одинаковой глубины, у каждого уровня есть родитель	Год → Квартал → Месяц → День
Несбалансированная	ветви разной глубины, но каждый узел — один и тот же тип сущности	оргструктура «начальник → подчинённый»
Рваная (ragged)	уровни фиксированы, но в некоторых ветвях пропущен промежуточный уровень	Страна → Штат → Город, где у города-государства нет штата
Сетевая	у элемента несколько родителей	товар в двух категориях

Разница несбалансированной и рваной: в несбалансированной *нет фиксированных уровней* (глубина произвольна), в рваной уровни заданы, но отдельные значения уровня отсутствуют.

3. Схемы «звезда», «снежинка», «созвездие»

3.1. Звезда

- **Таблица фактов** — в центре; длинная и узкая: внешние ключи на измерения и числовые меры.
- **Таблицы измерений** — вокруг; короткие и широкие: ключ и описательные атрибуты. Иерархия денормализована в одной таблице (Категория → Подкатегория → Товар в таблице «Товар»).

Зерно таблицы фактов определяется набором её измерений и смыслом строки.

3.2. Снежинка и созвездие

- **Снежинка** — измерения нормализованы: «Товар» ссылается на «Подкатегорию», та — на «Категорию». Экономит место, но добавляет соединения (JOIN) и усложняет модель для пользователя.
- **Созвездие (галактика)** — несколько таблиц фактов (Продажи, Остатки) используют общие **конформные** измерения (Время, Товар). Отсюда операция drill-across — переход между фактами.

3.3. Типы таблиц фактов

Тип	Строка — это	Особенность
Транзакционная	одно событие (позиция чека)	меры аддитивны, строки не обновляются
Периодический снимок	состояние на дату (остаток на конец дня)	меры полуаддитивны
Накапливающийся снимок	процесс с вехами (заказ: создан → оплачен → доставлен)	строка обновляется по мере прохождения вех
Безфактовая	только ключи, без мер	регистрация событий или покрытия (какие товары участвовали в акции)

3.4. Суррогатные ключи

Суррогатный ключ — целочисленный идентификатор, который генерирует само хранилище и который не несёт бизнес-смысла. Зачем он нужен:

1. хранить историю (одному натуральному ключу соответствуют несколько версий строки, см. SCD-2);
2. защититься от переиспользования кодов в источнике;
3. ускорить соединения (сравнивать целые числа быстрее, чем строки).

Неизвестные значения не кодируют `NULL` (строка пропадёт при соединении), а заводят служебные строки измерения: -1 «Неизвестно», -2 «Неприменимо».

> **В ЛР1.** На ER-диаграмме используются суррогатные INT-ключи. В Power BI вы связываете таблицы по натуральным ключам (`Customer ID`, `Product ID`) — для учебной модели это допустимо.

3.5. Связи в Power BI как математика

Связь `Superstore[Product ID]` → `Товар[Product ID]` с кратностью $* : 1$ — это функция $\varphi : \text{Факты} \to \text{Товары}$: каждой строке факта сопоставлена ровно одна строка справочника. Поэтому:

- `RELATED(Товар[Category])` для строки факта i возвращает $\mathrm{Category}(\varphi(i))$ — это значение функции;
- фильтр по измерению $A \subseteq \text{Товары}$ превращается в фильтр фактов $\varphi^{-1}(A) = \{i : \varphi(i) \in A\}$ — **прообраз** множества. Именно так фильтр «течёт» от «1» к «*».

Если ключ в справочнике не уникален, φ перестаёт быть функцией — отсюда ошибка «многие ко многим».

4. Медленно меняющиеся измерения (SCD)

Атрибуты измерений меняются (клиент переехал, товар переклассифицировали), а факты уже записаны. Вопрос: к какой версии атрибута относить старые факты?

Тип	Что делаем	История
0	никогда не меняем (дата регистрации)	исходное значение навсегда
1	перезаписываем	теряется
2	добавляем новую строку с новым суррогатным ключом и датами действия `valid_from`, `valid_to`	полная
3	храним «текущее» и «предыдущее» значение в двух столбцах	одна предыдущая версия
6 (1+2+3)	новая строка (тип 2) + столбец текущего значения во всех версиях (тип 1/3)	полная + удобный текущий срез

Технически предпочтительного решения нет, потому что выбор зависит от *смысла* вопроса. Финансисту нужна «правда на момент сделки» (тип 2), категорийному менеджеру — «всё в текущей структуре» (тип 1). Решение принимает **владелец данных**; оно фиксируется в бизнес-метаданных и спецификации ETL, причём отдельно для каждого атрибута.

5. OLAP: операции и архитектура

5.1. Операции навигации

Операция	Что делает	Размерность результата
Срез (slice)	фиксирует одно значение одного измерения (Регион = Москва)	уменьшается на 1 (3D → 2D)
Вырезание (dice)	ограничивает несколько измерений диапазонами (Урал + Сибирь, Q3)	**сохраняется** (под-куб той же размерности)
Вращение (pivot)	меняет оси отображения	не меняется; данные не меняются
Свёртка (roll-up)	подъём по иерархии (Город → Регион)	агрегирование
Детализация (drill-down)	спуск по иерархии	раскрытие агрегата
Drill-through	переход к исходным строкам фактов	ограничен зерном
Drill-across	переход к другому кубу по общим измерениям	требует конформных измерений

Разница среза и вырезания через размерность: срез даёт сечение размерности $n-1$, вырезание — подкуб размерности n. Вращение данных не меняет, но помогает увидеть закономерность, скрытую в другой раскладке (сезонность видна в столбцах, но не видна в строках).

5.2. Агрегаты и предагрегация

Агрегат — заранее посчитанная свёртка мер по некоторому набору уровней измерений. **Предагрегация** — расчёт агрегатов заранее (обычно ночью), чтобы отвечать на запросы за доли секунды.

Число возможных агрегатов. Пусть у измерения i есть h_i уровней иерархии плюс уровень «Всего». Агрегат задаётся выбором одного уровня в каждом измерении, поэтому

$$N_{\text{агр}} = \prod_{i=1}^{n} (h_i + 1).$$

Пример из лекции: Время (День, Месяц, Квартал, Год) $h = 4$, Товар (Товар, Подкатегория, Категория) $h = 3$, Магазин (Магазин, Город, Регион) $h = 3$, Клиент (Клиент, Сегмент) $h = 2$:

$$N = 5 \cdot 4 \cdot 4 \cdot 3 = 240.$$

Это **комбинаторный взрыв**: с ростом числа измерений агрегатов становится экспоненциально много, поэтому строят не все, а только частые, остальные выводят из них. Агрегаты образуют **решётку**: (Категория, Месяц) вычисляется из (Категория, День), но не наоборот.

Для вашей звезды Superstore: Календарь (Дата, Месяц, Квартал, Год) $h = 4$; Товар (Товар, Подкатегория, Категория) $h = 3$; Местоположение (Город, Штат, Регион, Страна) $h = 4$; Клиент (Клиент, Сегмент) $h = 2$; Способ доставки $h = 1$:

$$N = 5 \cdot 4 \cdot 5 \cdot 3 \cdot 2 = 600.$$

5.3. MOLAP, ROLAP, HOLAP

	MOLAP	**ROLAP**	**HOLAP**
Детали	в многомерном массиве	в реляционных таблицах	в реляционных таблицах
Агрегаты	в массиве	в таблицах (материализованные представления)	в массиве
Скорость	максимальная	зависит от СУБД	высокая на верхних уровнях
Объём	ограничен	терабайты	высокий

Современная колоночная СУБД (ClickHouse) стирает границы: она хранит детали в таблицах, но сканирует их со скоростью, близкой к MOLAP.

5.4. Тест FASMI

Fast (простые запросы < 5 с), **A**nalysis (бизнес-логика без программирования), **S**hared (многопользовательский доступ с правами), **M**ultidimensional (пользователь мыслит измерениями), **I**nformation (нет искусственных лимитов объёма).

Время отклика оценивают по процентилям (p95, p99), а не по среднему: 90 запросов по 1 с и 10 по 20 с дают среднее 2,9 с, но каждый десятый запрос — 20 секунд ожидания.

6. ETL, ELT и CDC

6.1. Этапы ETL

1. **Extract** — извлечение из источников (полное или инкрементальное).
2. **Transform** — очистка, приведение типов, дедупликация, обогащение, расчёт ключей, применение SCD.
3. **Load** — загрузка в хранилище (вставка, UPSERT/MERGE, обновление агрегатов).

ELT — сначала загрузить сырьё в мощную колоночную СУБД, затем трансформировать SQL-ом внутри неё. Сырые данные при этом сохраняются для переобработки.

6.2. Инкрементальная загрузка и CDC

Водяной знак (high-water mark): забирать строки, у которых `updated_at > :last_watermark`. Ловушки: граница сравнения (`>` против `>=`), рассинхронизация часовых поясов, изменения «задним числом». Слепое пятно — **удаления**: удалённую строку запрос по метке времени не увидит.

Метод CDC	Вмешательство в источник	Видит удаления	Нагрузка
По столбцу времени	нет	нет	низкая
Триггерный	изменение схемы	да	высокая (на каждую транзакцию)
Снимковый (diff)	нет	да	высокая (сравнение массивов)
Журнальный (WAL, binlog)	нет	**все изменения**	минимальная

Преимущество журнального CDC (например, Debezium): читает журнал транзакций СУБД, видит вставки, обновления и удаления, не нагружает источник и превращает пакетный ETL в непрерывный поток событий (например, в Kafka).

6.3. Идемпотентность и оркестрация

Идемпотентность — повторный запуск даёт тот же результат: $f(f(x)) = f(x)$. Наивный `INSERT` при перезапуске упавшей загрузки задублирует строки; `UPSERT/MERGE` («есть ключ — обнови, нет — вставь») идемпотентен.

DAG (направленный ациклический граф) — вершины-задачи, рёбра-зависимости. Ацикличность гарантирует, что есть порядок выполнения (топологическая сортировка) и конечная точка. **Оркестратор** (Apache Airflow) запускает задачи в нужном порядке, распараллеливает независимые ветви, повторяет упавшие и ведёт журнал.

6.4. Качество данных

Правило **1–10–100**: дефект стоит \$1 при вводе, \$10 при очистке в хранилище и \$100, если по нему приняли неверное решение. Показатели качества (ГОСТ Р ИСО/МЭК 25012): полнота, точность, согласованность, актуальность, уникальность, доступность. Каждый из них — измеримая доля, например

$$\text{Полнота} = \frac{\#\{\text{заполненных обязательных значений}\}}{\#\{\text{обязательных значений}\}}.$$

7. DAX: контексты вычисления

- **Контекст строки** — вычисление «для текущей строки» (вычисляемый столбец, итераторы `SUMX`, `AVERAGEX`, `ADDCOLUMNS`). В нём работает `RELATED`.
- **Контекст фильтра** — набор фильтров от визуала, срезов и связей. Мера вычисляется заново в каждом контексте фильтра.

Итератор $\texttt{SUMX}(T, e) = \sum_{r \in T} e(r)$ — сумма выражения по строкам. Отсюда разница:

$$\texttt{SUMX}(T, a \cdot b) = \sum_r a_r b_r \;\neq\; \texttt{SUM}(a) \cdot \texttt{SUM}(b) = \Big(\sum_r a_r\Big)\Big(\sum_r b_r\Big).$$

`AVERAGEX` пропускает пустые значения: $\frac{1}{|T'|}\sum_{r \in T'} e(r)$, где T' — строки с непустым e. Это видно в вашем запросе 1: средний срок пополнения считается только по товарам, у которых есть поставщик.

8. Вопросы для самопроверки (РК1)

1. Четыре свойства хранилища — назовите и раскройте каждое. (§1.1)
2. Чем staging отличается от ODS? (§1.2)
3. Шесть критериев сравнения подходов Инмона и Кимбалла. (§1.3)
4. Какие метаданные отсутствуют чаще всего и почему это опасно? (§1.4)
5. Почему куб — логическая модель? Почему тип данных не определяет роль поля? (§2.1–2.2)
6. Что такое зерно? Сформулируйте принцип невосстановимости детализации. (§2.3)
7. По два примера аддитивных, полуаддитивных и неаддитивных мер. (§2.4)
8. Чем несбалансированная иерархия отличается от рваной? (§2.5)
9. Звезда против снежинки; что такое созвездие? (§3)
10. Почему у SCD нет технически лучшего решения и кто его принимает? (§4)
11. Посчитайте число агрегатов для куба с измерениями из 3, 2 и 4 уровней. Ответ: $4 \cdot 3 \cdot 5 = 60$. (§5.2)
12. Срез против вырезания через размерность; зачем нужно вращение? (§5.1)
13. Этапы ETL; преимущество журнального CDC; что такое DAG? (§6)